



DATA CORRUPTING ARCHITECTURES

WE KNOW AND LOVE



SEAN T. ALLEN

VP OF ENGINEERING @ WALLAROO LABS
MEMBER OF THE PONY CORE TEAM
AUTHOR OF “STORM APPLIED”
CHICHARRONES LOVER

@SEANTALLEN
@WALLAROO LABS
@PONYLANG

TODAY'S TOPICS INCLUDE...

- ▶ Data corruption
- ▶ Concurrency
- ▶ Data races
- ▶ Shared values
- ▶ Atomic operations



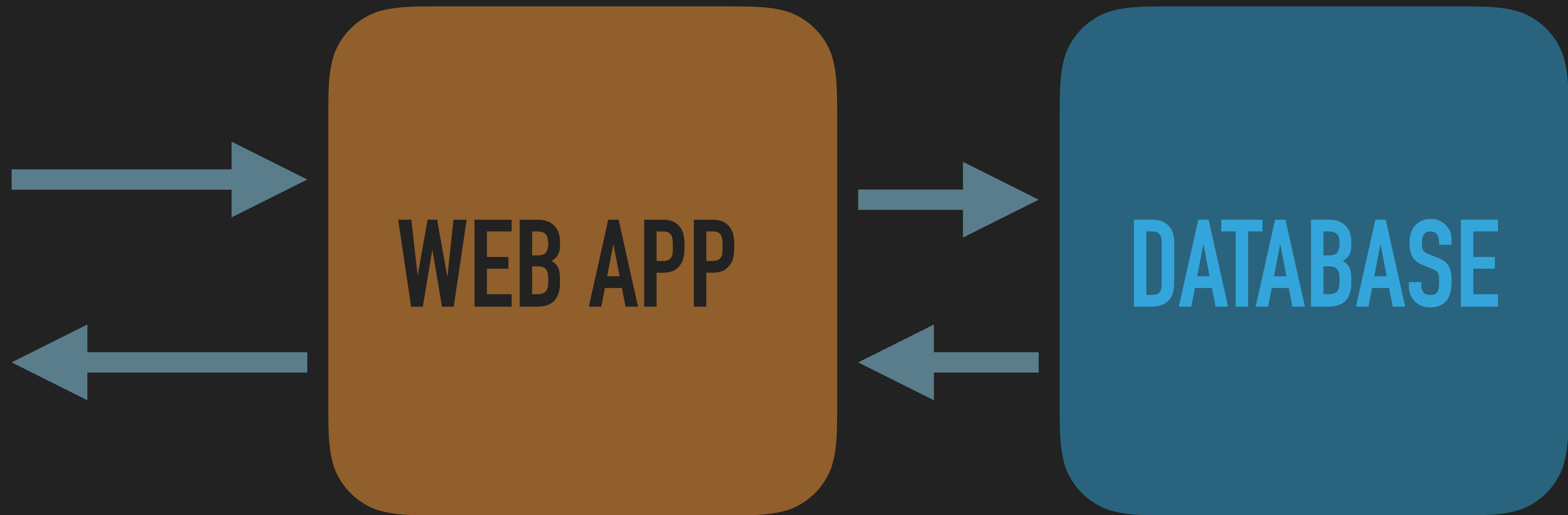
DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE

GENERALIZATIONS AHEAD

**OUR MOST POPULAR ARCHITECTURES
CAN BE TRACED TO WEB
APPLICATIONS.**

Sean T. Allen

DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE



DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE

A BIT MORE DETAILED...

REQUEST

to the

APPLICATION

on to the

DATABASE

back to the

APPLICATION

and out as a

RESPONSE



WE COULD GET MORE DETAILED STILL....

REQUEST

to the

APPLICATION

on to the

DATABASE

back to the

APPLICATION

and out as a

RESPONSE

(WEB BROWSER)

to the

(WEB SERVER + DEVELOPER CODE)

on to the

DATABASE

back to the

(WEB SERVER + DEVELOPER CODE)

and out as a

(WEB BROWSER)



OUR
SIMPLE WEB APP IS
A DISTRIBUTED SYSTEM

**DISTRIBUTED SYSTEMS AREN'T HARD.
THEY'RE EASY*.**

Sean T. Allen

*** sometimes**

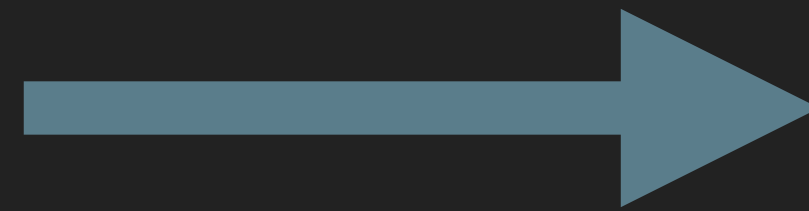
**DISTRIBUTED SYSTEMS AREN'T HARD.
THEY'RE EASY*.
CONCURRENCY IS THE HARD PART.**

Sean T. Allen

* sometimes

SOME JUSTIFICATION AHEAD

WEB APP



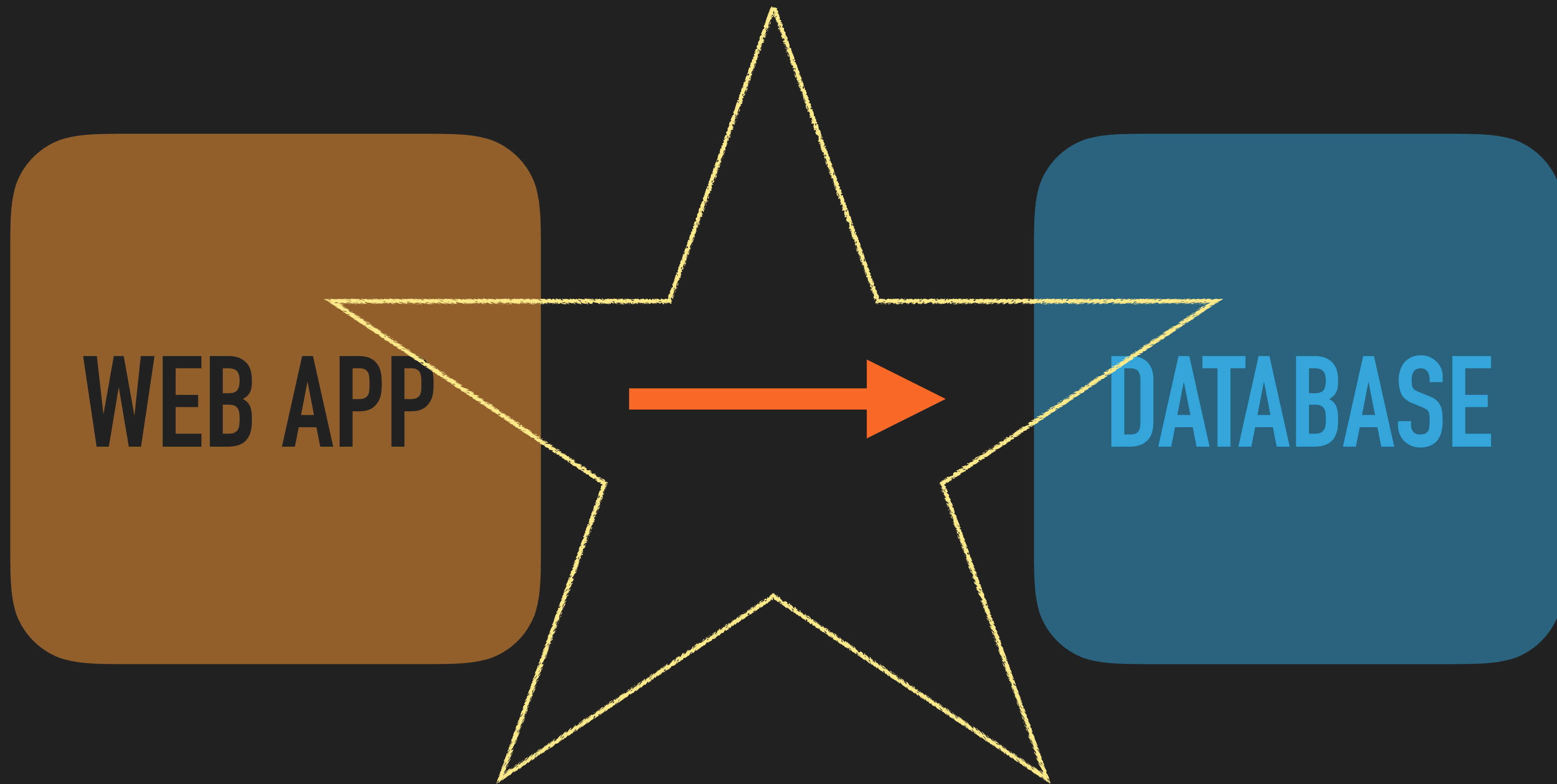
DATABASE

WEB APP



DATABASE

DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE



AN EXAMPLE APPLICATION

THE PAGES SERVED COUNTER

- ▶ Read current value from the database
- ▶ Add one to update it
- ▶ Write new value to the database
- ▶ Return new value to the user



THE PAGES SERVED COUNTER

- ▶ Read current value from the database
- ▶ Add one to update it
- ▶ Write new value to the database
- ▶ Return original value to the user

**SPOT THE
DATA CORRUPTION?**



THE PAGES SERVED COUNTER

- ▶ Read current value from the database
- ▶ Add one to update it
- ▶ Write new value to the database
- ▶ Return new value to the user



THE PAGES SERVED COUNTER

- ▶ Read current value from the database
 - ▶ Add one to update it
 - ▶ Write new value to the database
 - ▶ Return original value to the page
- # LET'S ADD SOME CONCURRENCY



THE PAGES SERVED COUNTER

- ▶ Read current value from the database
- ▶ Add one to update it
- ▶ Write new value to the database
- ▶ Return new value to the user

- ▶ Read current value from the database
- ▶ Add one to update it
- ▶ Write new value to the database
- ▶ Return new value to the user

THE PAGES SERVED COUNTER

WHAT'S THE FINAL VALUE?

- ▶ Read current value from the database
- ▶ Add one to update it
- ▶ Write new value to the database
- ▶ Return original value to the user

1?
2?

DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE

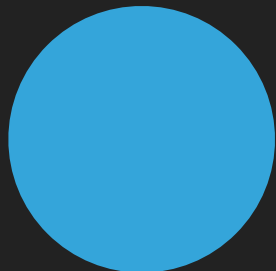
**WE DON'T
KNOW**

THE PAGES SERVED COUNTER

- ▶ Read current value from the database
- ▶ Add one to update it
- ▶ Write new value to the database
- ▶ Return new value to the user

- ▶ Read current value from the database
- ▶ Add one to update it
- ▶ Write new value to the database
- ▶ Return new value to the user

DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE



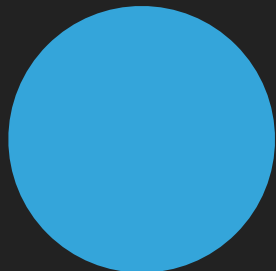
Request A



Request B

▶ Read current value from the database	0
▶ Read current value from the database	0
▶ Add one to update it	1
▶ Add one to update it	1
▶ Write new value to the database	1
▶ Write new value to the database	1
▶ Return new value to the user	1
▶ Return new value to the user	1

DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE



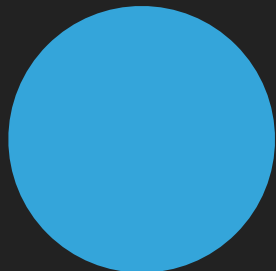
Request A



Request B

▶ Read current value from the database	0
▶ Add one to update it	1
▶ Write new value to the database	1
▶ Return new value to the user	1
▶ Read current value from the database	1
▶ Add one to update it	2
▶ Write new value to the database	2
▶ Return new value to the user	2

DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE



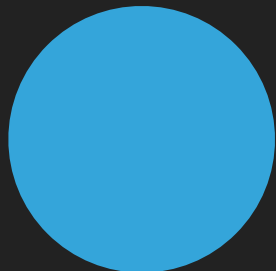
Request A



Request B

▶ Read current value from the database	0
▶ Add one to update it	1
▶ Write new value to the database	1
▶ Read current value from the database	1
▶ Return new value to the user	1
▶ Add one to update it	2
▶ Write new value to the database	2
▶ Return new value to the user	2

DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE



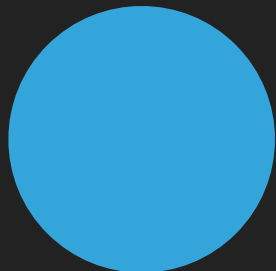
Request A



Request B

▶ Read current value from the database	0
▶ Add one to update it	1
▶ Write new value to the database	1
▶ Read current value from the database	1
▶ Return new value to the user	1
▶ Add one to update it	2
▶ Write new value to the database	2
▶ Return new value to the user	2

DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE



Request A



Request B

▶ Read current value from the database	0
▶ Add one to update it	1
▶ Write new value to the database	1
▶ Read current value from the database	1
▶ Return new value to the user	1
▶ Add one to update it	2
▶ Write new value to the database	2
▶ Return new value to the user	2

WE'RE EXPERIENCING A DATA RACE

RECIPE FOR A DATA RACE

- ▶ A shared value
- ▶ Concurrency
- ▶ At least one writer changing the shared value
- ▶ Non-atomic operations



**OUR DATABASE IS
A SHARED VALUE**

CONCURRENCY:

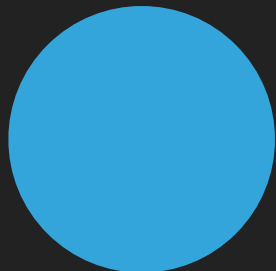
WE HAVE TWO REQUESTS

WHOLE LOTTA
WRITING
GOING ON

OUR INCREMENTING ISN'T ATOMIC

AN OPERATION IS **ATOMIC** IF IT'S
ISOLATED FROM OTHER
OPERATIONS HAPPENING AT THE
SAME TIME.

DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE



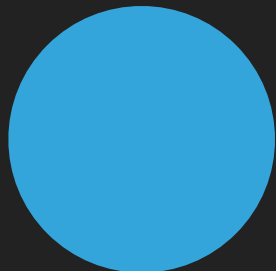
Request A



Request B

▶ Read current value from the database	0
▶ Read current value from the database	0
▶ Add one to update it	1
▶ Add one to update it	1
▶ Write new value to the database	1
▶ Write new value to the database	1
▶ Return new value to the user	1
▶ Return new value to the user	1

DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE



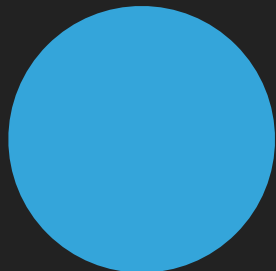
Request A



Request B

▶ Read current value from the database	0
▶ Read current value from the database	0
▶ Add one to update it	1
▶ Add one to update it	1
▶ Write new value to the database	1
▶ Write new value to the database	1
▶ Return new value to the user	1
▶ Return new value to the user	1

DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE



Request A



Request B

▶ Increment current value

1

▶ Increment current value

2

▶ Read new value

2

▶ Read new value

2

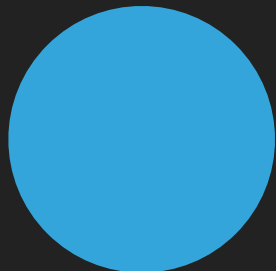
▶ Return new value to the user

2

▶ Return new value to the user

2

DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE



Request A



Request B

▶ Increment current value

1

▶ Increment current value

2

▶ Read new value

2

▶ Read new value

2

▶ Return new value to the user

2

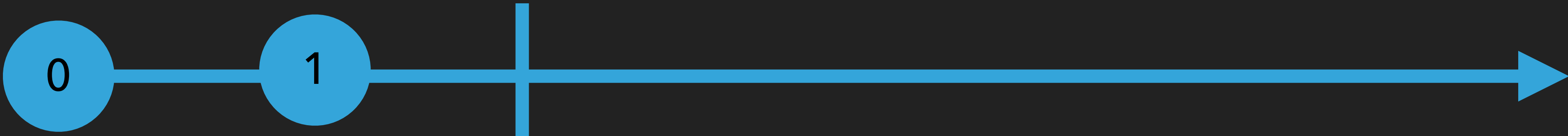
▶ Return new value to the user

2

MORE CONCURRENCY
MORE PROBLEMS

DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE

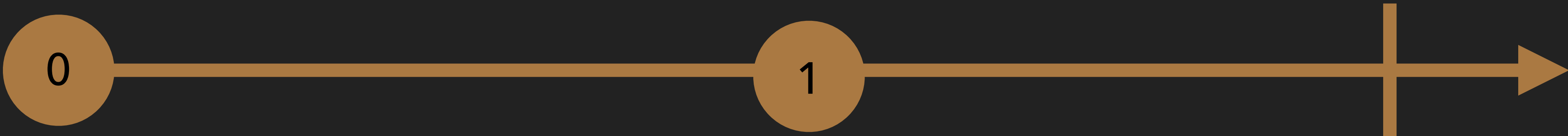
Request A



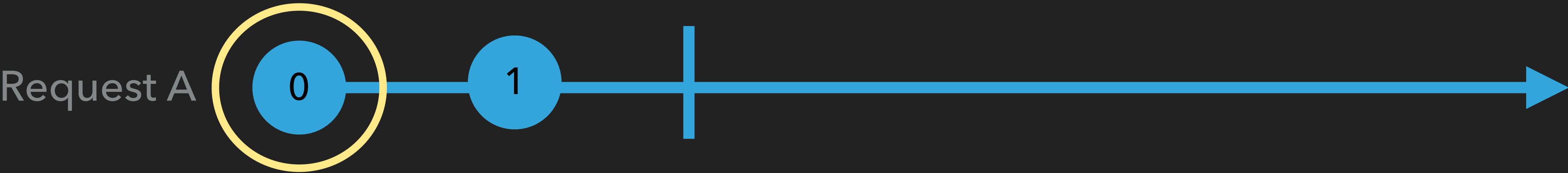
Request B



Request C

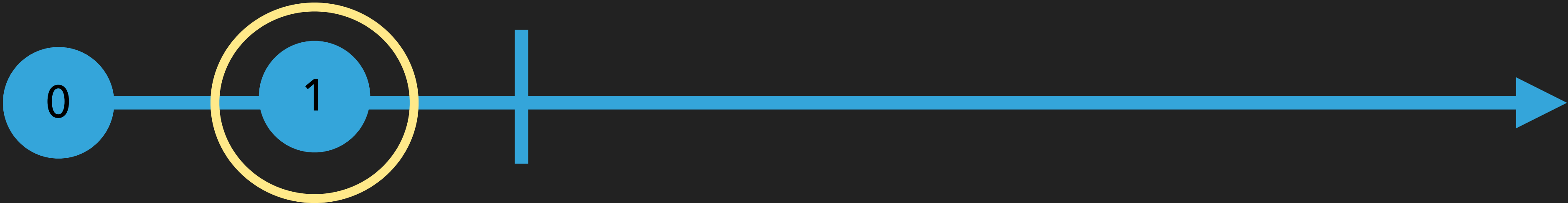


DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE



DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE

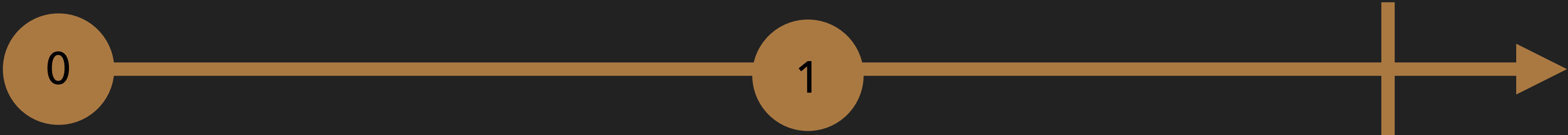
Request A



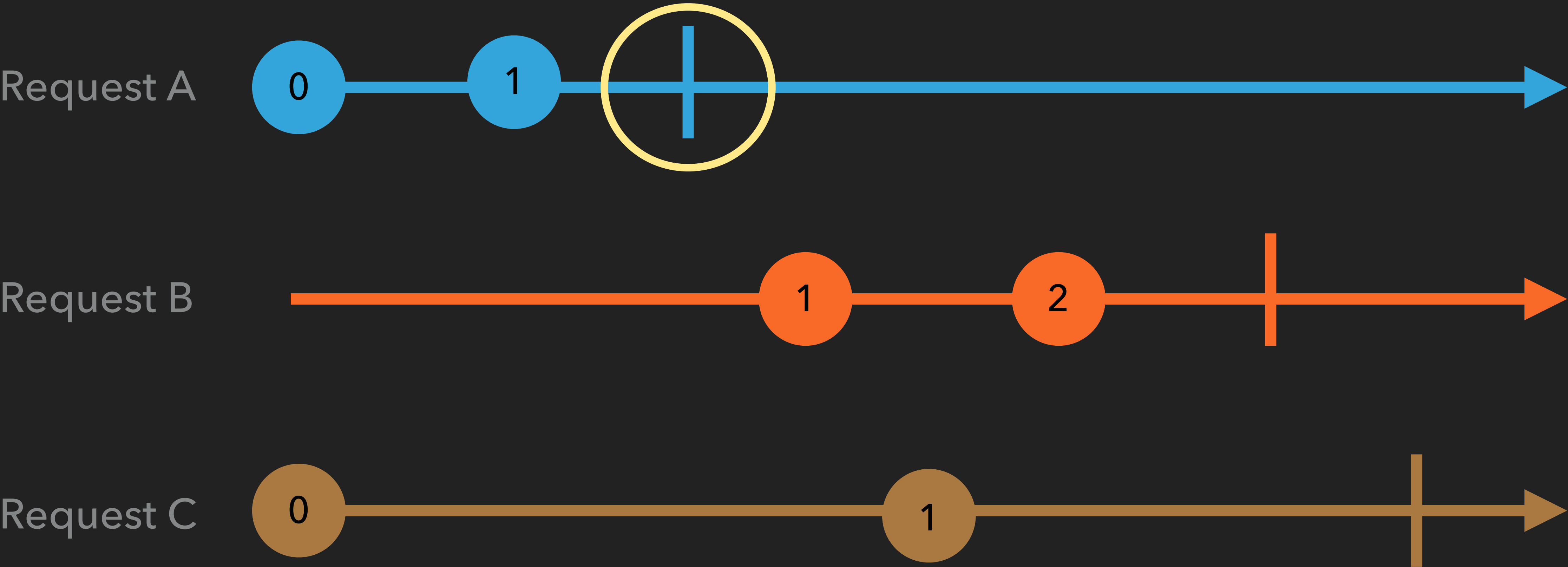
Request B



Request C



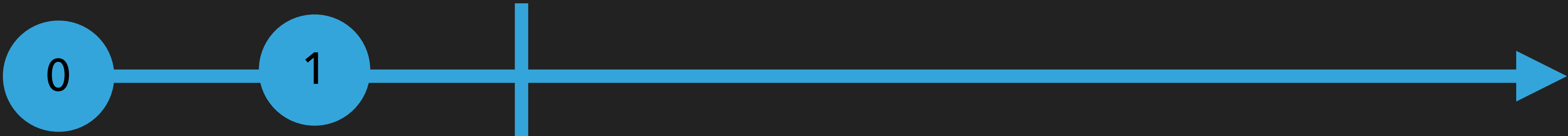
DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE



DATABASE VALUE IS 1

DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE

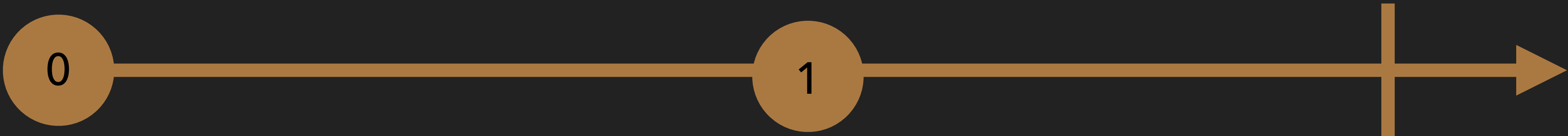
Request A



Request B

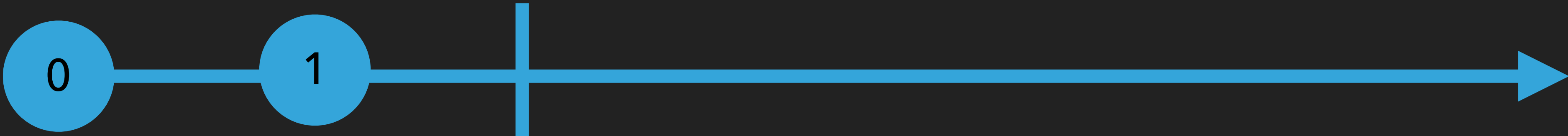


Request C

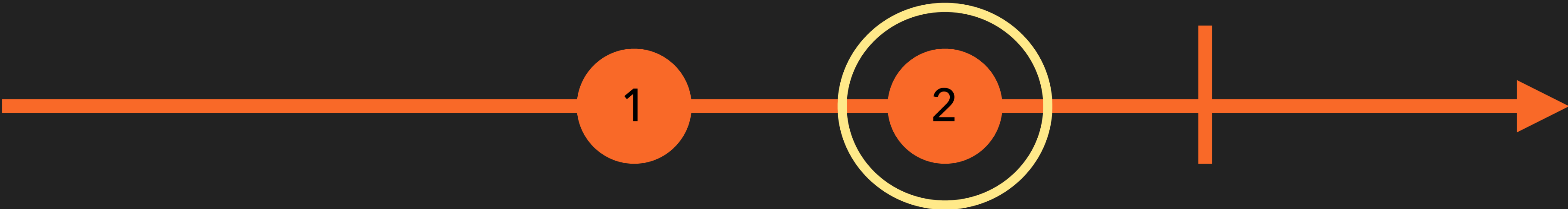


DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE

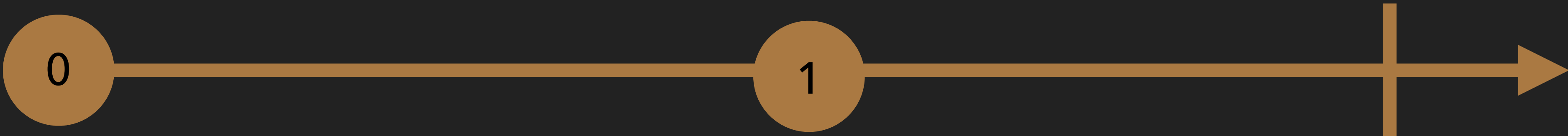
Request A



Request B

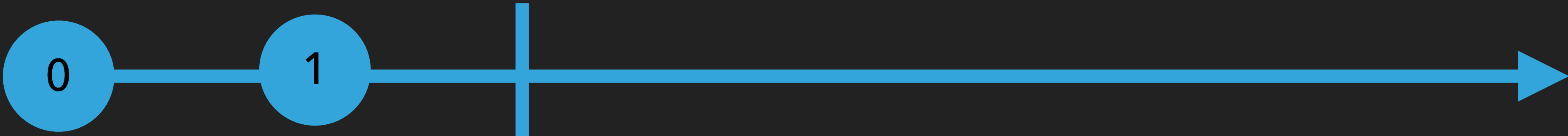


Request C

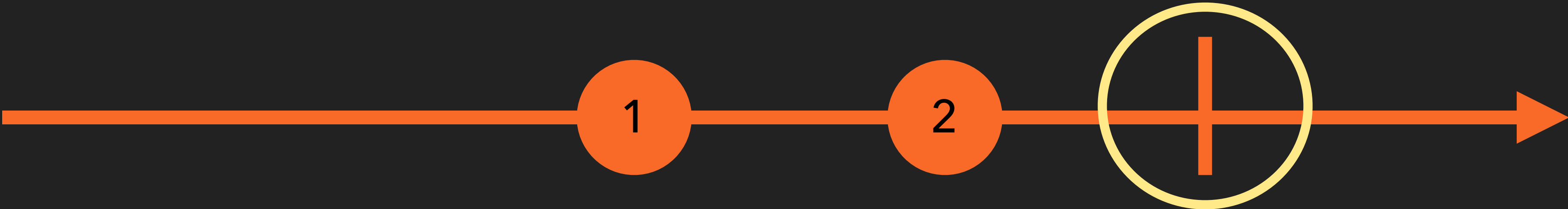


DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE

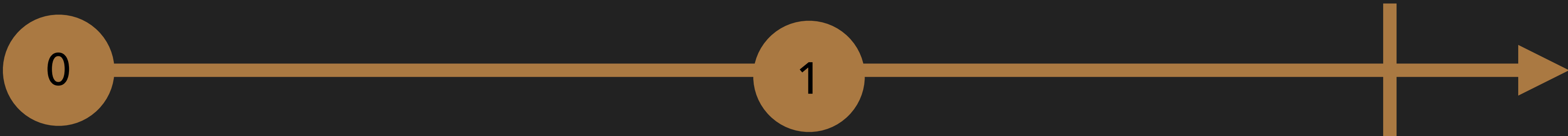
Request A



Request B



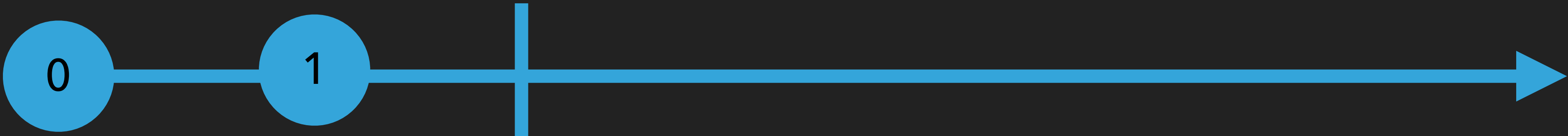
Request C



**DATABASE VALUE
INCREASED TO 2**

DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE

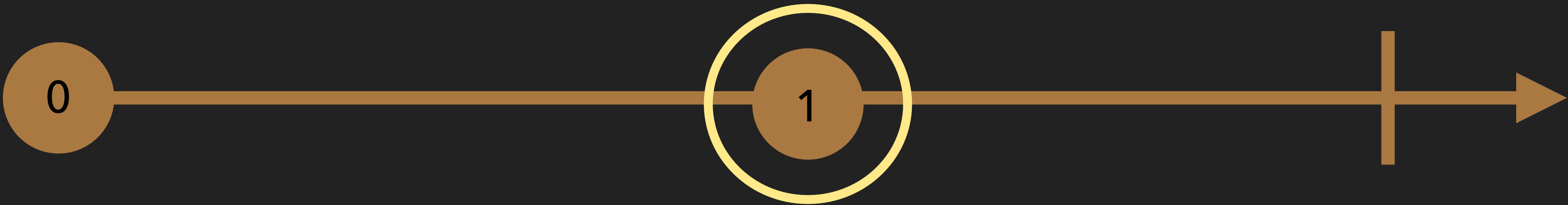
Request A



Request B

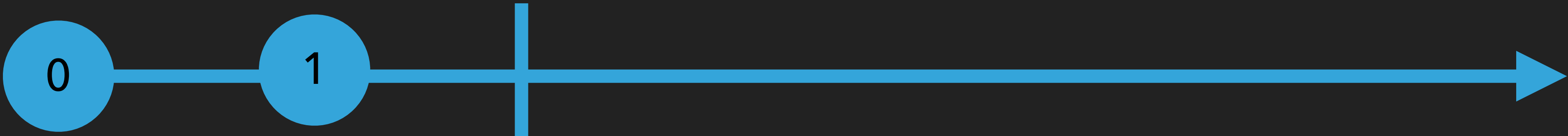


Request C



DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE

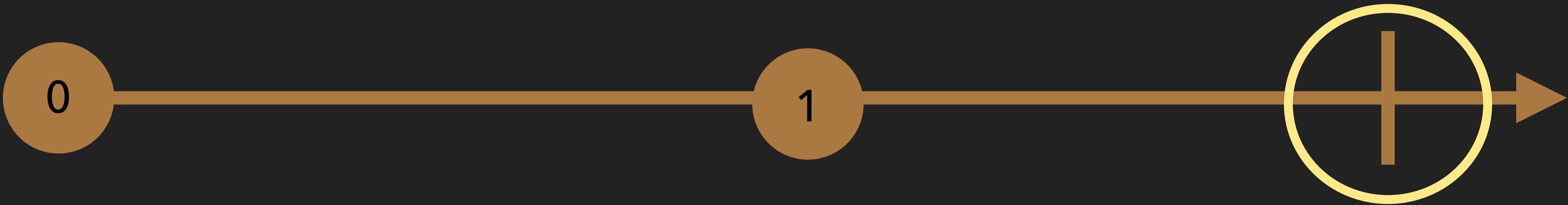
Request A



Request B



Request C



**DATABASE VALUE
DECREASED TO 1**

DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE

**WE'RE EXPERIENCING
LAST WRITE WINS**

LAST WRITE WINS MEANS “SOME WRITES LOSE”.

Sean Cribbs & Russell Brown

**LAST WRITE WINS IS A FANCY NAME
FOR “LET’S CORRUPT SOME DATA”.**

Sean T. Allen

DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE

OUR MOST POPULAR ARCHITECTURES FOR SCALING

THESE PROBLEMS
ARE EVERYWHERE

“SERVERLESS”

“FUNCTIONS AS SERVICE”

“STATELESS”

DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE

“SHARED NOTHING”

ALL FEATURE DATABASE AS A SHARED VALUE AND CONCURRENT WRITES

**ALL HAVE
DATA CORRUPTION
BAKED INTO THE CORE ARCHITECTURE**

**WHAT
CAN WE DO?**

RECIPE FOR A DATA RACE: CHANGE AT LEAST ONE

- ▶ A shared value
- ▶ Concurrency
- ▶ At least one writer changing the shared value
- ▶ Non-atomic operations



STORED PROCEDURES AND TRANSACTIONS

DATA CORRUPTING ARCHITECTURES WE KNOW AND LOVE

DISTRIBUTED TRANSACTIONS

SYNCHRONIZE DATA ACCESS

CONSENSUS ALGORITHMS

CONCURRENCY

FRIENDLY DATA STRUCTURES

COMMUNICATE
OPERATIONS
NOT
VALUES



LEARN MORE

[GITHUB.COM/
SEANTALLEN/DATA-
CORRUPTING-
ARCHITECTURES](https://github.com/seantallen/data-corrupting-architectures)